TP : Mission AES - Crack the Secret!

Imane Fouad, UM6P

TOP SECRET — YOUR MISSION

Following your successful decryption of the mysterious archive, the CC@UM6P security team intercepts a new message. The attackers have learned from their mistakes: Caesar and Vigenère are gone, replaced by **AES-128**. A warning accompanies the transmission: *"You broke our old ciphers by exploiting patterns. Let's see if you can break this one."* Your mission: explore the inner workings of AES, uncover the secrets of **confusion and diffusion**, reverse its transformations, and discover whether modern encryption is truly unbreakable.

Rules: Work by hand in Act I (no AI, no code). In Act II, type and complete the provided code; record both *what you found* and *why it works*.

Learning Goals

By the end of this lab, students should be able to:

- understand the principles of modern symmetric encryption and the structure of AES-128;
- explain the concepts of confusion and diffusion and their importance in cryptographic security;
- understand and apply the main AES transformations: SubBytes, ShiftRows, MixColumns, and AddRoundKey;
- identify security weaknesses of ECB mode and explain why strong encryption algorithms can still be used insecurely.

ACT I — THE AES TRAINING ROOM

In your previous mission, you broke classical ciphers by exploiting patterns. CryptoVault has now replaced them with AES-128. Before accessing its encrypted archive, you must understand how the new encryption engine works. Complete the following puzzles to unlock the vault terminal.

RECALL — WHAT YOU NEED

Unlike Caesar and Vigenère, which operate on letters, AES operates on **fixed-size blocks of bytes**. AES is a **symmetric block cipher**: the same secret key is used for encryption and decryption.
AES-128 uses:

- a block size of 128 bits (16 bytes);
- a secret key of 128 bits (16 bytes);
- 10 encryption rounds.

Each 16-byte block is represented internally as a 4×4 matrix called the **state**.
Throughout this training room, we use the standard AES matrix layout, with each entry written in hexadecimal.

The intercepted block is represented by the following state:

$$S = \begin{bmatrix} 00 & 01 & 02 & 03 \\ 10 & 11 & 12 & 13 \\ 20 & 21 & 22 & 23 \\ 30 & 31 & 32 & 33 \end{bmatrix}$$

RECALL — WHAT YOU NEED

In your previous lab, Caesar used a numerical shift and Vigenère used a repeating secret keyword. AES also uses a secret key, but combines it with the state differently.
The **AddRoundKey** transformation combines every state byte with the corresponding round-key byte using bitwise **XOR** ($\oplus$).

$$S' = S \oplus K$$

Remember the XOR rules:

$$0 \oplus 0 = 0, \qquad 1 \oplus 1 = 0$$

$$0 \oplus 1 = 1, \qquad 1 \oplus 0 = 1$$

An interesting property is that XOR reverses itself:

$$(x \oplus k) \oplus k = x$$

AES-128 derives 11 round keys from the original secret key. AddRoundKey is performed before the first round and at the end of each round.
It is the transformation that directly introduces the **secret key** into the AES state.

A CryptoVault technician left the following simplified round key for testing:

$$K = \begin{bmatrix} 0F & 0F & 0F & 0F \\ 0F & 0F & 0F & 0F \\ 0F & 0F & 0F & 0F \\ 0F & 0F & 0F & 0F \end{bmatrix}$$

Your mission:

1. Apply AddRoundKey to the entire state S from Puzzle 1.

$$S' = S \oplus K = \begin{bmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{bmatrix}$$

RECALL — WHAT YOU NEED

Think back to Caesar!
In the Caesar cipher, encryption follows a simple rule:

$$C = (P + k) \bmod 26$$

This relationship is predictable, which makes the cipher easy to attack.
Modern encryption must make the relationship between the **plaintext, ciphertext, and secret key** much more difficult to analyze.
This idea is called **confusion**, a principle introduced by Claude Shannon.
AES introduces strong nonlinearity using the **SubBytes** transformation.
Each state byte is replaced by another byte using a predefined lookup table called the **S-box**.
For example:

$$\texttt{00} \longrightarrow \texttt{63}$$

Unlike Caesar's simple shift, the AES S-box is designed to create a complex, nonlinear transformation.
Important: SubBytes changes byte values, but does not move them to different positions.

The security team recovered the following extract from the actual AES S-box:

Input	00	01	02	03
Output	63	$7C$	77	$7B$

Your mission:

2 Apply SubBytes to the first row of the original state S:

$$[00,\ 01,\ 02,\ 03] \longrightarrow [\square,\ \square,\ \square,\ \square]$$

RECALL — WHAT YOU NEED

Confusion alone is not enough!
Imagine that AES only applied SubBytes. Each byte would be substituted independently, but a change in one byte would not directly affect the other positions.
An attacker could potentially study each byte position separately.
This is why AES also needs **diffusion**.
Diffusion aims to spread the influence of each input byte across many output bytes.
AES achieves this by combining two transformations:

- **ShiftRows**: rearranges byte positions;
- **MixColumns**: mixes byte values within columns.

Together, these transformations help spread changes throughout the state over successive AES rounds.
In ShiftRows:

- Row 0 is unchanged.
- Row 1 is rotated left by 1 position.
- Row 2 is rotated left by 2 positions.
- Row 3 is rotated left by 3 positions.

The shifts are **cyclic**: bytes leaving one side reappear on the other side.

Return to the original state:

$$S = \begin{bmatrix} 00 & 01 & 02 & 03 \\ 10 & 11 & 12 & 13 \\ 20 & 21 & 22 & 23 \\ 30 & 31 & 32 & 33 \end{bmatrix}$$

Your mission:

3 Apply ShiftRows and complete the resulting matrix.

$$\text{ShiftRows}(S) = \begin{bmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{bmatrix}$$

RECALL — WHAT YOU NEED

ShiftRows rearranges bytes, but does not actually combine their values.
The **MixColumns** transformation takes the next step: it mixes the four bytes within each column using arithmetic in the finite field $GF(2^8)$.
As a result, changing one input byte can affect all four output bytes of that column.
The transformation uses a fixed matrix:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

All operations take place in $GF(2^8)$, not ordinary integer arithmetic.
For this exercise, remember:

- Addition is performed using XOR.
- Multiplication by `02` uses a left shift; if the original most significant bit was 1, XOR with `1B` after retaining 8 bits.
- Multiplication by `03` is:

$$(03 \cdot x) = (02 \cdot x) \oplus x$$

Why does this matter?
ShiftRows moves bytes between columns, while MixColumns combines bytes within each column. Over successive rounds, their combined effect provides diffusion across the AES state.

The security team intercepted the following column:

$$\begin{bmatrix} D4 \\ BF \\ 5D \\ 30 \end{bmatrix}$$

The first output byte is calculated as:

$$b_0 = (02 \cdot D4) \oplus (03 \cdot BF) \oplus 5D \oplus 30$$

Your mission:

4 Calculate the final byte b_0.

RECALL — WHAT YOU NEED

You have now explored the four transformations that form the core of AES:

- **SubBytes**: nonlinear substitution;
- **ShiftRows**: rearrangement of bytes;
- **MixColumns**: mixing of bytes;
- **AddRoundKey**: combination with a round key.

AES repeats these transformations over multiple rounds to achieve strong encryption. The final encryption round omits MixColumns.

AES without Diffusion

CryptoVault has removed both MixColumns and ShiftRows from its encryption algorithm. As a result, each byte is encrypted independently. The CC@UM6P security team has intercepted three plaintext/ciphertext pairs. Can you recover the secret keys?

We consider a simplified cipher with only **two rounds**, operating on a single byte.

The encryption process is:

$$p \xrightarrow{\oplus k_0} \xrightarrow{S} \xrightarrow{\oplus k_1} \xrightarrow{S} \xrightarrow{\oplus k_2} c$$

Here, S is the AES S-box and k_0, k_1, k_2 are three unknown key bytes.

Intercepted plaintext/ciphertext pairs:

Plaintext p	Ciphertext c
`00`	`A4`
`01`	`F8`
`02`	`3F`

Your mission:

1. Write the encryption equation relating p, c, and the three key bytes.
2. Using the first pair, express k_2 as a function of k_0 and k_1.
3. Write a small program to recover all key candidates compatible with the three intercepted pairs (check Appendix for S box).
4. Explain why removing both MixColumns and ShiftRows makes this attack possible.

ACT II — THE FINAL VAULT: REVERSE THE CIPHER

The AES training room is complete. Your team has unlocked the CryptoVault terminal and discovered the final encrypted transmission. Unlike the classical ciphers you broke in the previous lab, this message is protected by AES-128. Fortunately, the security team has recovered the secret key—but the decryption engine is incomplete.

Your final mission: rebuild the AES decryption process and recover the hidden message.

RECALL — WHAT YOU NEED

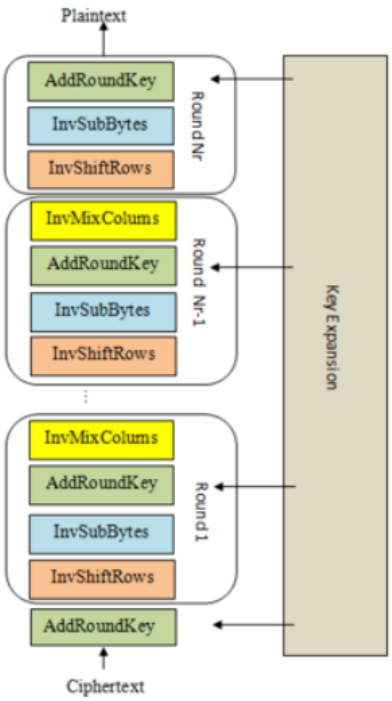
In Act I, you explored the four main AES transformations: SubBytes, ShiftRows, MixColumns, and AddRoundKey.
These operations work together to provide **confusion**, **diffusion**, and key-dependent encryption.
AES-128 applies 10 encryption rounds using 11 round keys derived from the original secret key.
To decrypt a message, the transformations must be undone in the appropriate reverse order.
Each transformation has an inverse:

- **InvSubBytes:** Uses the inverse S-box to recover each byte before substitution.
- **InvShiftRows:** Rotates the rows to the right instead of the left.
- **InvMixColumns:** Uses an inverse matrix to undo the mixing of the four bytes within each column.
- **AddRoundKey:** Applies XOR with the same round key, since $(x \oplus k) \oplus k = x$.

The round keys are applied in **reverse order**. Remember that the final encryption round does not include MixColumns.

Mission — Rebuild the Decryption Engine

Study the AES encryption/decryption diagram provided below.



The security team recovered the following evidence:

Objective. Recover a secret 16-byte ASCII message from one encrypted AES-128 block. Implement the inverse AES transformations the *key expansion is already provided.*

Your tasks:

1. Implement `add_round_key(state, round_key)` using XOR.
2. Implement `inv_shift_rows(state)` (right rotations by 0, 1, 2 and 3 positions).
3. Implement `inv_sub_bytes(state)` using `INV_SBOX`.
4. Implement `gf_mul(a,b)` using `xtime()` and XOR.
5. Implement `inv_mix_columns(state)` with the supplied 4×4 inverse matrix.
6. Complete `decrypt_block(ciphertext, round_keys)` and display the hidden text.

Decryption order: initial AddRoundKey with K_{10}; for rounds $9, \ldots, 1$, InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ AddRoundKey $\rightarrow$ InvMixColumns; finally InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ AddRoundKey with K_0.

Starter code

```
# AES S-box (public standard table). For decryption, invert this mapping.
SBOX = [int(x, 16) for x in """63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76
ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0
b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15
04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75
09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84
53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf
d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8
51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2
cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73
60 81 4f dc 22 2a 90 88 46 ee b8 14 de 5e 0b db
e0 32 3a 0a 49 06 24 5c c2 d3 ac 62 91 95 e4 79
e7 c8 37 6d 8d d5 4e a9 6c 56 f4 ea 65 7a ae 08
ba 78 25 2e 1c a6 b4 c6 e8 dd 74 1f 4b bd 8b 8a
70 3e b5 66 48 03 f6 0e 61 35 57 b9 86 c1 1d 9e
e1 f8 98 11 69 d9 8e 94 9b 1e 87 e9 ce 55 28 df
8c a1 89 0d bf e6 42 68 41 99 2d 0f b0 54 bb 16
""".split()]
INV_SBOX = [0] * 256
for index, value in enumerate(SBOX):
    INV_SBOX[value] = index

# First byte of each Rcon word, for the 10 AES-128 expansions.
RCON = [0x01, 0x02, 0x04, 0x08, 0x10,
        0x20, 0x40, 0x80, 0x1B, 0x36]


def expand_key(key):
    """PROVIDED: expand a 16-byte AES key into 11 round keys.

    Every 4-byte word is a column. w[0]..w[3] are the original key.
    At positions divisible by four: RotWord, SubWord, Rcon, then XOR.
    At all positions: XOR with the word four positions earlier.
    Output: round_keys[round][row][column] for rounds 0..10.
    """
    if len(key) != 16:
        raise ValueError("AES-128 requires exactly 16 key bytes")
    words = [list(key[4*i : 4*i+4]) for i in range(4)]
    for i in range(4, 44):
        temp = words[i-1].copy()
        if i % 4 == 0:
            temp = temp[1:] + temp[:1]            # RotWord
```

```
            temp = [SBOX[b] for b in temp]          # SubWord
            temp[0] ^= RCON[i//4-1]                 # Rcon
        words.append([words[i-4][j] ^ temp[j] for j in range(4)])

    round_keys = []
    for rnd in range(11):
        cols = words[4*rnd : 4*rnd+4]
        round_keys.append([[cols[c][r] for c in range(4)]
                           for r in range(4)])
    return round_keys


def bytes_to_state(block):
    """TO DO."""



def state_to_bytes(state):
    """TO DO"""


def xtime(value):
    """PROVIDED: multiply by 02 in GF(2^8), reducing with 0x1B."""
    return ((value << 1) & 0xFF) ^ (0x1B if value & 0x80 else 0)

# =====  IMPLEMENT THE INVERSE AES TRANSFORMATIONS =====

def add_round_key(state, round_key):
    """TODO 1: XOR every state byte with the matching round-key byte.
    Both arguments are 4x4 matrices, indexed as [row][column].
    Modify state in place.
    """


def inv_shift_rows(state):
    """TODO 2: circularly shift row r RIGHT by r positions.
    Row 0: 0, row 1: 1, row 2: 2, row 3: 3 positions.
    """



def inv_sub_bytes(state):
    """TODO 3: substitute each byte using the provided INV_SBOX."""



def gf_mul(a, b):
    """TODO 4: multiply two bytes in GF(2^8).
    HINT: when b's low bit is 1, XOR a into the result;
    update a with xtime(a); shift b one bit right; repeat.
    """



def inv_mix_columns(state):
    """TODO 5: transform every column using the inverse AES matrix:

        0E 0B 0D 09
        09 0E 0B 0D
        0D 09 0E 0B
        0B 0D 09 0E

    Multiply with gf_mul, combine using XOR. Modify state in place.
    """
```

```
def decrypt_block(ciphertext, round_keys):
    """TODO 6: AES-128 inverse cipher for exactly one 16-byte block.
    Suggested sequence:
      1. Build the 4x4 state from ciphertext.
      2. AddRoundKey using round key 10.
      3. For rounds 9 down to 1:
         InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.
      4. InvShiftRows, InvSubBytes, AddRoundKey using round key 0.
      5. Convert the final state to bytes and return it.
    """


# ===== PROVIDED TEST INPUT:  =====
KEY = bytes.fromhex("2b7e151628aed2a6abf7158809cf4f3c")
CIPHERTEXT = bytes.fromhex("d21bb1d61866796323d61a23c635fb45")

if __name__ == "__main__":
    keys = expand_key(KEY)
    assert len(keys) == 11
    plaintext = decrypt_block(CIPHERTEXT, keys)
    print("Plaintext (hex):", plaintext.hex())
    print("Secret message: ", plaintext.decode("ascii"))
```

MISSION COMPLETE — CRYPTOVAULT UNLOCKED

APPENDIX — The Vault's AES Tool Kit

Appendix — AES S-box (Python)

The AES S-box is a substitution table containing 256 values. Each input byte is replaced by the corresponding value in the table.

For example:

$$S(00) = 63, \qquad S(03) = 7B.$$

The substitution is performed using `s_box[x]`, where `x` is an integer between 0 and 255.

```
# AES Substitution Box (S-box)
# Usage: s_box[0x03] returns 0x7B

s_box = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5,
    0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0,
    0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc,
    0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a,
    0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0,
    0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b,
    0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85,
    0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5,
    0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17,
    0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88,
    0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c,
    0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9,
    0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6,
    0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e,
    0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94,
    0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68,
    0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
]
```

Example:

```
print(hex(s_box[0x00]))  # 0x63
print(hex(s_box[0x03]))  # 0x7b
print(hex(s_box[0x71]))  # 0xa3
```

Appendix — AES Inverse Transformations

A.1 — Inverse AES S-box

During decryption, InvSubBytes uses the inverse AES S-box to recover the original byte values.

If $S(x) = y$, then $S^{-1}(y) = x$.

For example:

$$S(\texttt{00}) = \texttt{63} \quad \Longrightarrow \quad S^{-1}(\texttt{63}) = \texttt{00}.$$

To find the inverse substitution, use the first hexadecimal digit of the input as the row and the second digit as the column.

invsbox = [0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb, 0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb, 0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e, 0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25, 0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65, 0xb6, 0x92, 0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46, 0x57, 0xa7, 0x8d, 0x9d, 0x84, 0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06, 0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b, 0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6, 0x73, 0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8, 0x1c, 0x75, 0xdf, 0x6e, 0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b, 0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4, 0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec, 0x5f, 0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93, 0xc9, 0x9c, 0xef, 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61, 0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d]

Inverse MixColumns matrix All values are hexadecimal. Multiplication is performed in GF(2^8).

$\text{invmix}_m atrix = [[0x0e, 0x0b, 0x0d, 0x09], [0x09, 0x0e, 0x0b, 0x0d], [0x0d, 0x09, 0x0e, 0x0b], [0x0b, 0x0d, 0x09, 0x0e]]$

All multiplications are performed in $GF(2^8)$, and additions correspond to XOR operations.

Example:

$$\begin{bmatrix} \texttt{04} \\ \texttt{66} \\ \texttt{81} \\ \texttt{E5} \end{bmatrix} \xrightarrow{\text{InvMixColumns}} \begin{bmatrix} \texttt{D4} \\ \texttt{BF} \\ \texttt{5D} \\ \texttt{30} \end{bmatrix}.$$